

Руководство по разработке плагинов для Minecraft (Paper) в VS Code

Часть 1: Подготовка рабочего окружения

Прежде чем писать код, необходимо настроить инструменты. Minecraft работает на Java, поэтому версия Java критически важна. Установка Java зависит от версии Minecraft.

- Minecraft 1.20.6 — 1.21.x: Требуется Java 21.
- Minecraft 1.18 — 1.20.4: Требуется Java 17.

Java: <https://www.oracle.com/europe/java/technologies/>

Выбор среды разработки. Создавать плагин можно где угодно, IntelliJ IDEA / VS Code. В данном гайде я буду использовать VS Code.

IntelliJ IDEA: <https://www.jetbrains.com/idea/download/>

VS Code: <https://code.visualstudio.com/download>

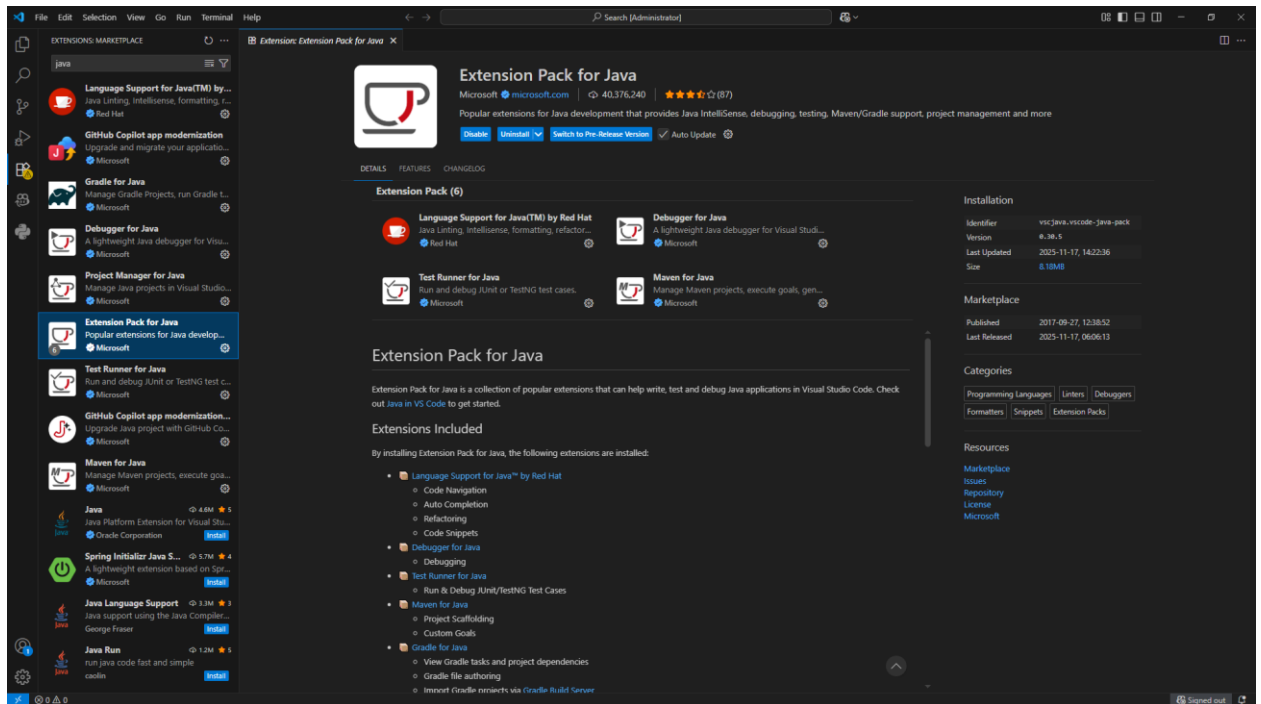
Настройка VS Code:

1. Откройте VS Code.
2. Перейдите во вкладку "Расширения" (Ctrl+Shift+X).
3. Введите в поиске: Extension Pack for Java.
4. Нажмите Install.

Система сборки Maven. Это инструмент, который:

- Автоматически скачивает библиотеки.
- Собирает проект в готовый .jar файл.
- Управляет версиями.
- Скачивать Maven не нужно, он встроен в расширение VS Code.

Также для сервера нужно скачать его ядро <https://papermc.io/downloads/paper>.



Часть 2: Создание проекта

В VS Code нажмите **Ctrl+Shift+P** (палитра команд) и введите **Java: Create Java Project -> Maven -> maven-archetype-quickstart**.

Выберите версию 1.4.

GroupId: com.example (это уникальная подпись, как домен сайта).

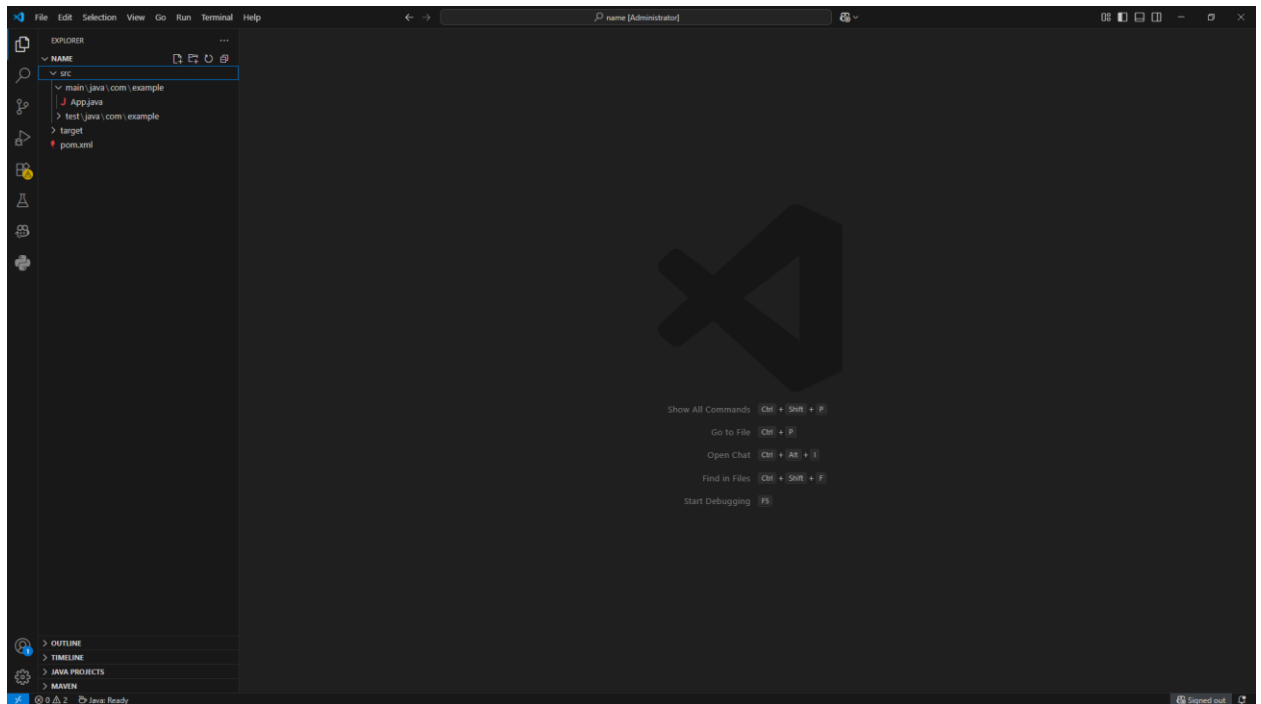
ArtifactId: Name (имя плагина).

Выберите папку на диске, где будет лежать проект.

Если проект не создается:

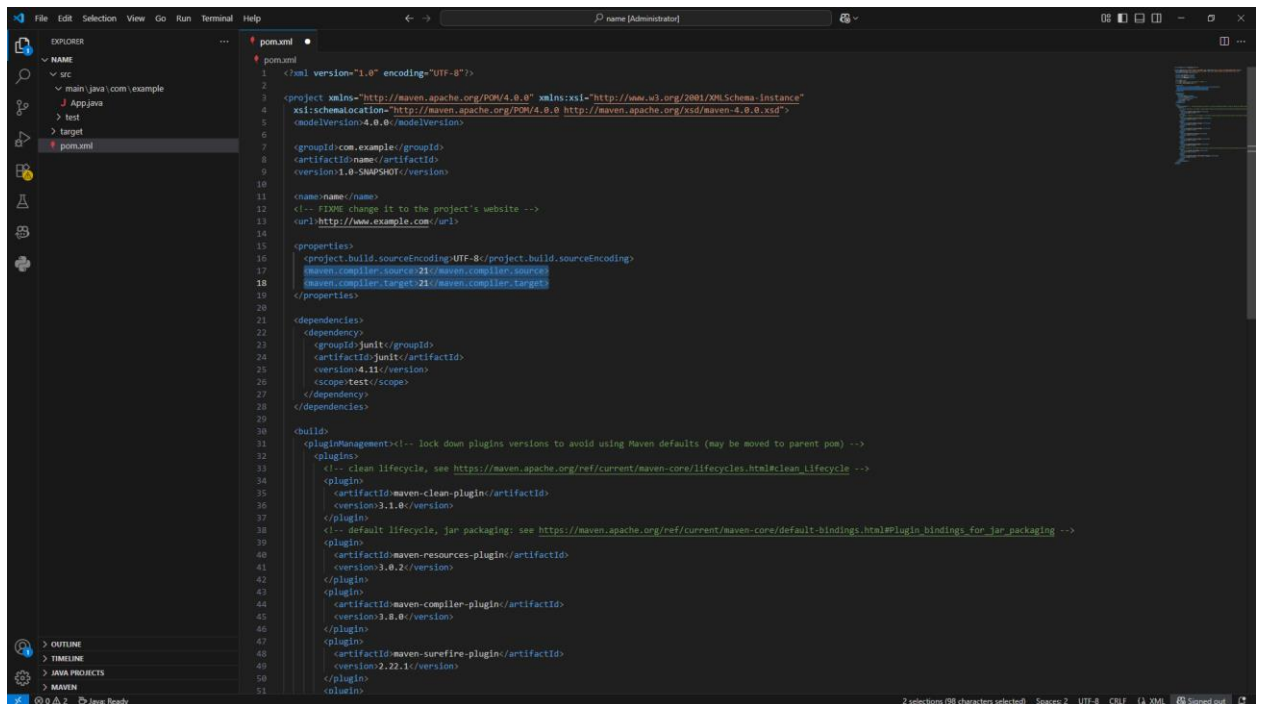
1. Откройте новый терминал (**Ctrl+Shift+`**)
2. Перейдите в папку проекта (**cd путь\к\проекту**)
3. Запустите сборку используя команду: `«mvn archetype:generate "-DgroupId=com.example" "-DartifactId=name" "-DarchetypeArtifactId=maven-archetype-quickstart" "-DarchetypeVersion=1.4" "-DinteractiveMode=false"»`

В терминале будет надпись **«BUILD SUCCESS»**. Откройте папку проекта.



Настройка Maven

Перейдите в pom.xml. В <properties> укажите версию Java.



```
<properties>  
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>  
  <maven.compiler.source>21</maven.compiler.source>  
  <maven.compiler.target>21</maven.compiler.target>  
</properties>
```

Также нужно указать репозиторий Paper и саму библиотеку, правильно подобрав версию:

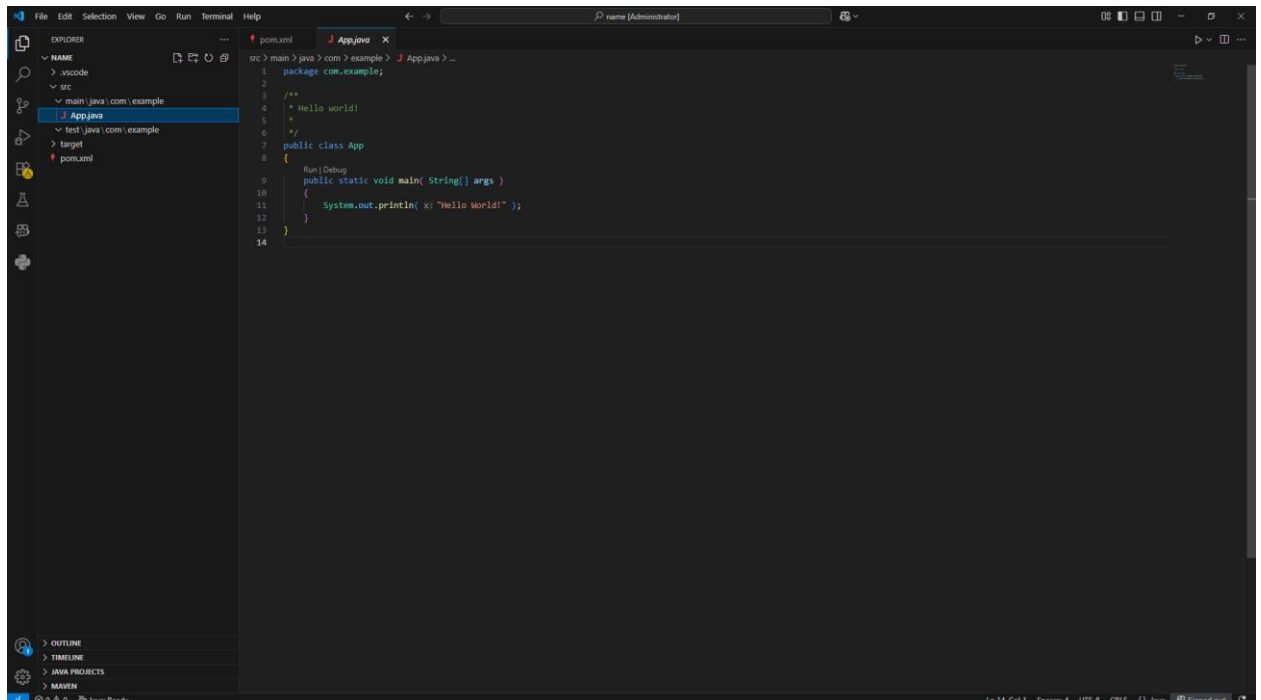
```
<repositories>
  <repository>
    <id>papermc-repo</id>
    <url>https://repo.papermc.io/repository/maven-public/</url>
  </repository>
</repositories>

<dependencies>
  <dependency>
    <groupId>io.papermc.paper</groupId>
    <artifactId>paper-api</artifactId>
    <version>1.21.4-R0.1-SNAPSHOT</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

Сохраните файл.

Часть 3: Написание кода

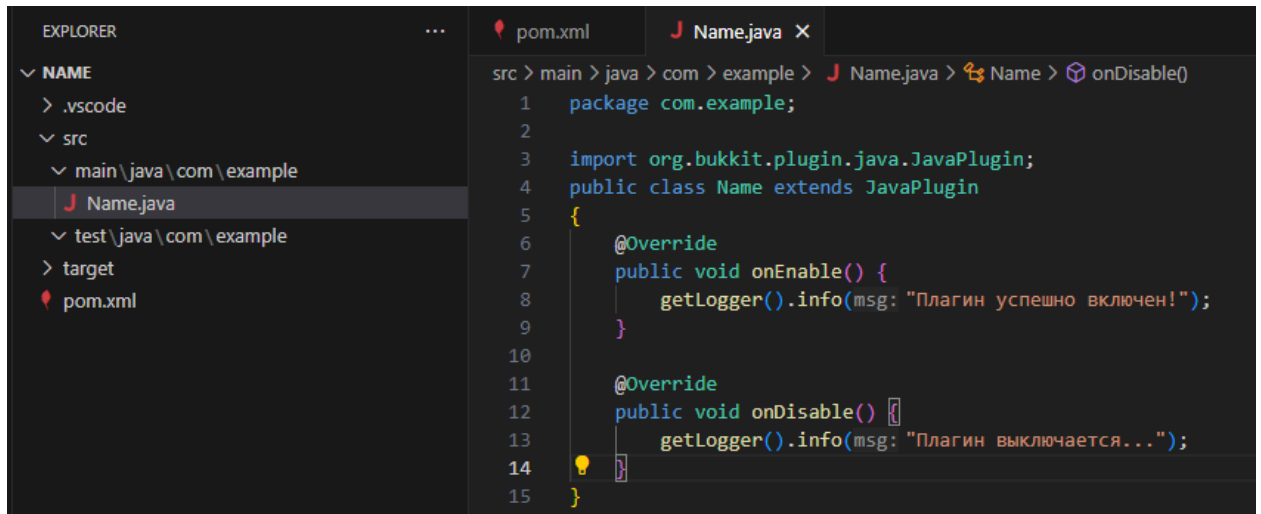
Откройте файл .java в src/main/java/com/example



Здесь и будет находиться ваш код плагина. Начнем:

Создадим две функции, первая будет работать когда сервер включается,, вторая когда сервер выключается.

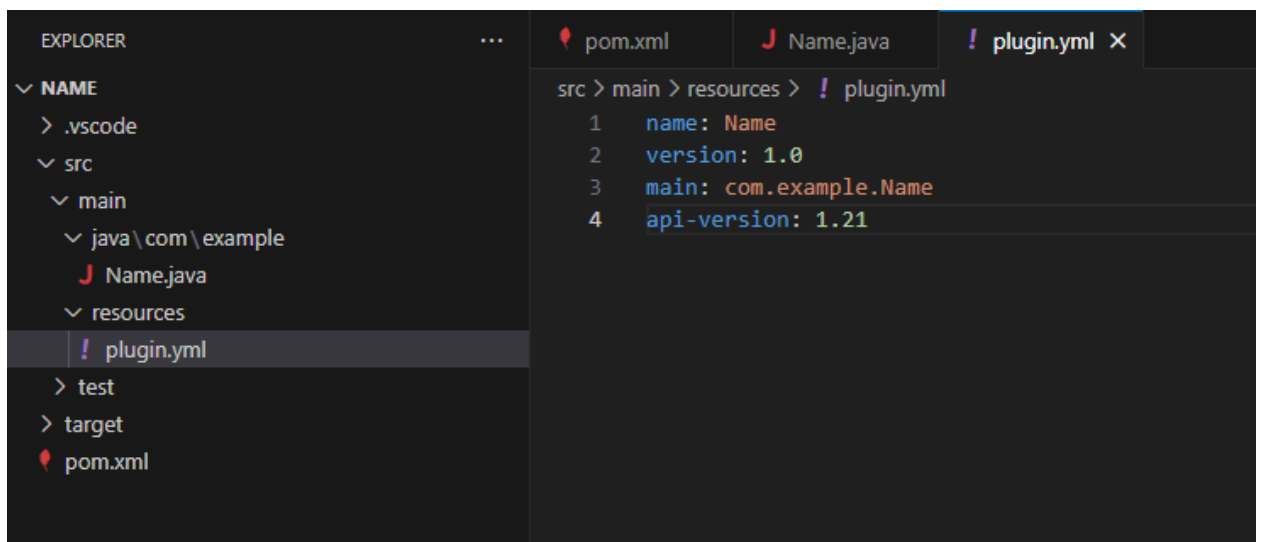
Добавим вывод какого-нибудь сообщения в консоль. Для работы нам понадобится `import org.bukkit.plugin.java.JavaPlugin;`



```
src > main > java > com > example > J Name.java > Name > onDisable()
1  package com.example;
2
3  import org.bukkit.plugin.java.JavaPlugin;
4  public class Name extends JavaPlugin
5  {
6      @Override
7      public void onEnable() {
8          getLogger().info(msg: "Плагин успешно включен!");
9      }
10
11     @Override
12     public void onDisable() {
13         getLogger().info(msg: "Плагин выключается...");
14     }
15 }
```

Также нужно настроить `plugin.yml` в `src/main/resources`. Если папки или файла нет просто создайте.

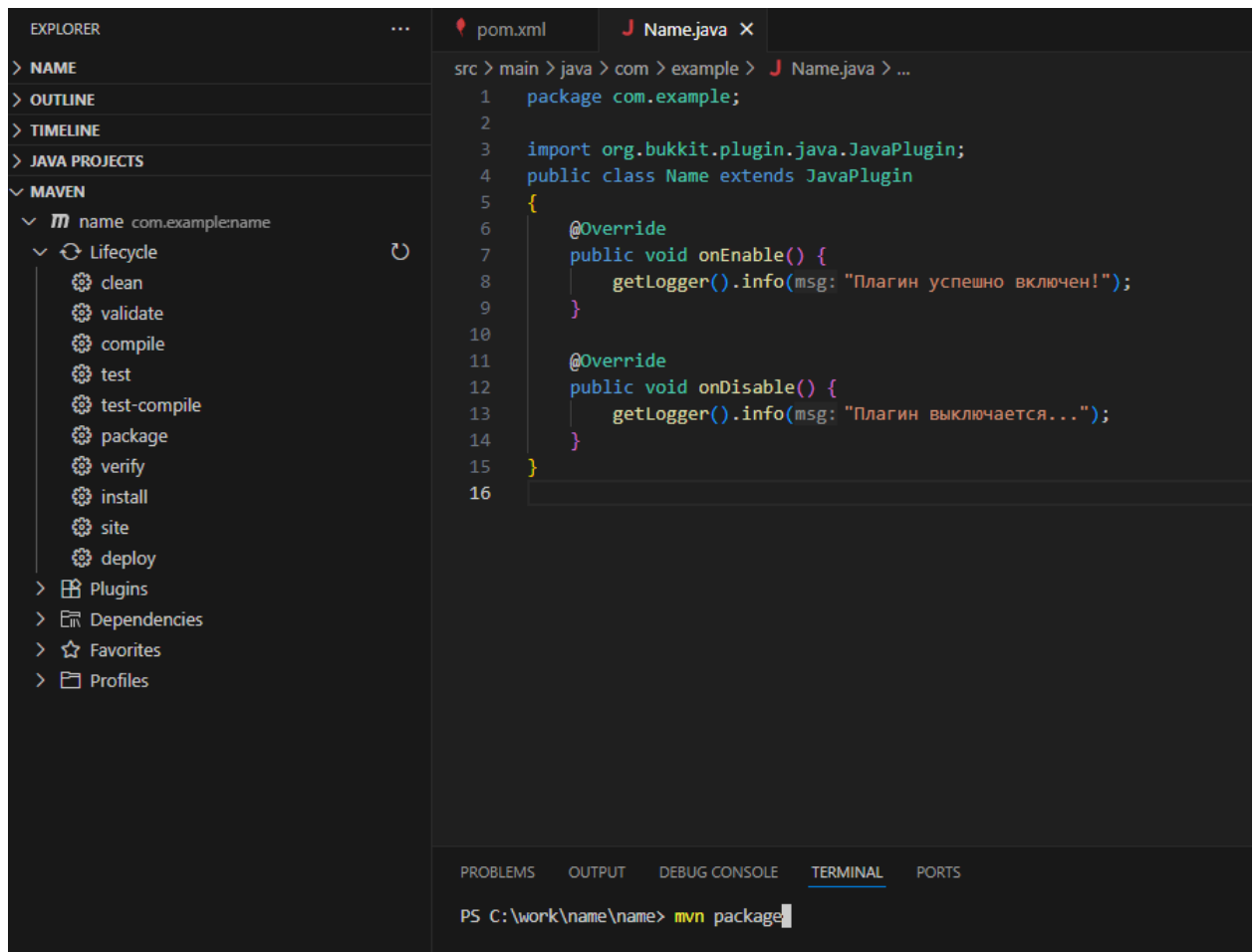
1. name – название плагина
2. version – версия плагина
3. main – groupId + ArtifactId
4. api-version – версия api



```
src > main > resources > ! plugin.yml
1  name: Name
2  version: 1.0
3  main: com.example.Name
4  api-version: 1.21
```

Можем проверить плагин. Нам нужно запустить сборку чтобы получить `.jar` файл для сервера. В Explorer раскрываем Maven. Переходим в Lifecycle и

нажимаем на треугольник у package. Либо можно запустить сборку в терминале, откроем его и пропишем «mvn package»



```

PS C:\work\name\name> mvn package
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:name >-----
[INFO] Building name 1.0-SNAPSHOT
[INFO]    from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.0.2:resources (default-resources) @ name ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory C:\work\name\name\src\main\resources
[INFO]
[INFO] --- compiler:3.8.0:compile (default-compile) @ name ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 1 source file to C:\work\name\name\target\classes
[INFO]
[INFO] --- resources:3.0.2:testResources (default-testResources) @ name ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory C:\work\name\name\src\test\resources
[INFO]
[INFO] --- compiler:3.8.0:testCompile (default-testCompile) @ name ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- surefire:2.22.1:test (default-test) @ name ---
[INFO]
[INFO] --- jar:3.0.2:jar (default-jar) @ name ---
[INFO] Building jar: C:\work\name\name\target\name-1.0-SNAPSHOT.jar
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time:  1.391 s
[INFO] Finished at: 2025-12-18T17:15:15+03:00
[INFO]
PS C:\work\name\name>

```

В папке target появится файл, его нужно переместить в папку plugins у нашего сервера и можно запускать.

```

[20:12:48 INFO]: Preparing spawn area: 0%
[20:12:48 INFO]: Time elapsed: 33 ms
[20:12:48 INFO]: [Name] Enabling Name v1.0
[20:12:48 INFO]: [Name] Плагин успешно включен!

```

Плагинами чаще всего пишут специальные команды и ивенты.

Продemonстрирую создание уникального предмета через команду. Создадим функцию для команд.

```

public boolean onCommand(CommandSender sender, Command cmd, String
label, String[] args)

```

```

1  package com.example;
2
3  import org.bukkit.command.Command;
4  import org.bukkit.command.CommandSender;
5  import org.bukkit.plugin.java.JavaPlugin;
6
7  public class Name extends JavaPlugin
8  {
9      @Override
10     public void onEnable() {
11         getLogger().info(msg: "Плагин успешно включен!");
12     }
13
14     @Override
15     public void onDisable() {
16         getLogger().info(msg: "Плагин выключается...");
17     }
18
19     @Override
20     public boolean onCommand(CommandSender sender, Command cmd, String label, String[] args) {
21
22         return false;
23     }
24
25 }
26

```

1. CommandSender – отправитель команды, может быть как игроком так и консолью
2. Command – сама команда
3. String label — точный текст команды
4. String[] args – это список аргументов, которые пишутся после команды, например /lorenote аргументы

Команда для получения уникального предмета подразумевает что игрок будет получать предмет, а значит что нет смысла писать команду через консоль.

Добавим проверку чтобы нашу команду делал именно игрок:

```

@Override
public boolean onCommand(CommandSender sender, Command cmd, String label, String[] args) {
    if (cmd.getName().equalsIgnoreCase("lorenote")) {
        if (!(sender instanceof Player)) {
            sender.sendMessage("Эту команду может использовать только игрок.");
            return true;
        }
    }
    return false;
}

```


Добавим проверку чтобы вместе с командой были аргументы. Для красивого текста можно изменять цвет с помощью ChatColor:

```
if (cmd.getName().equalsIgnoreCase("lorenote")) {  
    if (!(sender instanceof Player)) {  
        sender.sendMessage("Эту команду может использовать только игрок.");  
        return true;  
    }  
  
    if (args.length == 0) {  
        sender.sendMessage(ChatColor.RED + "Использование: /lorenote <текст>");  
        return true;  
    }  
}
```

Создадим переменную player, которая преобразуется из CommandSender в Player, и переменную message, которая хранит полученные аргументы. Создадим отдельную функцию, которая будет делать нам уникальный предмет с текстом.

Создадим новый предмет при помощи ItemStack. Получаем его мета данные для последующего редактирования. Изменяем название предмета meta.setDisplayName("");. Добавляем текст как описание предмета и применяем изменения:

```
private ItemStack createLoreNote(String text) {  
    ItemStack note = new ItemStack(Material.PAPER);  
    ItemMeta meta = note.getItemMeta();  
    meta.setDisplayName(ChatColor.GOLD + "Таинственная записка");  
    meta.setLore(Arrays.asList(text));  
    note.setItemMeta(meta);  
    return note;  
}
```

В onCommand добавим выполнение функции createLoreNote и выдадим игроку:

```

    if (args.length == 0) {
        sender.sendMessage(ChatColor.RED + "Использование: /lorenote <текст>");
        return true;
    }

    Player player = (Player) sender;
    String message = String.join(delimiter: " ", args);
    ItemStack item = createLoreNote(message);
    player.getInventory().addItem(item);
    player.sendMessage(ChatColor.GREEN + "Вы получили записку!");

    return true;
}

```

Также регистрируем команду в plugin.yml:

```

name: Name
version: 1.0
main: com.example.Name
api-version: 1.21
commands:
  lorenote:
    description: Выдает записку [d] текстом
    usage: /lorenote <текст>

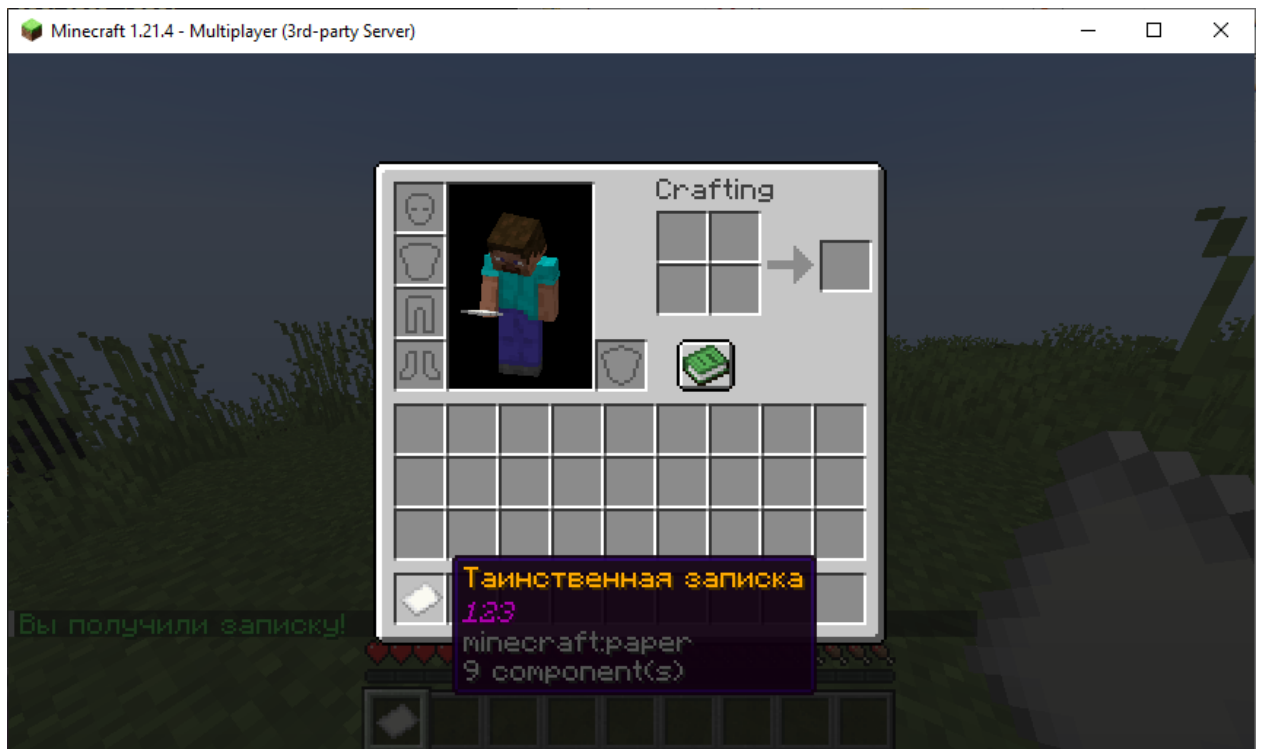
```

Собираем плагин и проверяем:

```

[20:51:06 INFO]: Done preparing level "world" (0.966s)
[20:51:06 INFO]: Running delayed init tasks
[20:51:06 INFO]: Done (6.817s)! For help, type "help"
> lorenote 123
[20:51:13 INFO]: Эту команду может использовать только игрок.
>

```



Теперь создадим ивент. Ивентом может быть все что происходит в Minecraft. Чтобы "услышать" ивент, класс должен реализовать интерфейс `Listener(implements Listener)`, а над методом-обработчиком нужно написать аннотацию `@EventHandler`. Регистрируем событие чтобы `onPortalCreate` работал:

```
public class Name extends JavaPlugin implements Listener
{
    @Override
    public void onEnable() {
        getServer().getPluginManager().registerEvents(this, this);
        getLogger().info(msg: "Плагин успешно включен!");
    }
}
```

Сделаем разрушение портала в нижний мир во время зажигания:

```
@EventHandler
public void onPortalCreate(PortalCreateEvent event) {
    if (event.getReason() != PortalCreateEvent.CreateReason.FIRE) return;
}
```

Сохраняем список блоков рамки. Добавим задержку в 1 тик чтобы плагин разрушил портал после его создания, иначе портал сохраняется, несмотря на то что мы плагином меняем блок рамки.

```
List<BlockState> blocks = new ArrayList<>(event.getBlocks());
new BukkitRunnable() {
    @Override
    public void run() {}
}.runTaskLater(this, 1L);
```

Находим в списке блок обсидиана и заменяем его, например на плачущий обсидиан.

```
new BukkitRunnable() {
    @Override
    public void run() {
        for (BlockState state : blocks) {
            if (state.getType() == Material.OBSIDIAN) {
                Block realBlock = state.getBlock();
                realBlock.setType(Material.CRYING_OBSIDIAN);
                break;
            }
        }
    }
}.runTaskLater(this, 1L);
```

Также можем добавить визуальный и звуковой эффект при помощи `.playSound` и `.spawnParticle` соответственно.

```
@EventHandler
public void onPortalCreate(PortalCreateEvent event) {
    if (event.getReason() != PortalCreateEvent.CreateReason.FIRE) return;
    List<BlockState> blocks = new ArrayList<>(event.getBlocks());
    new BukkitRunnable() {
        @Override
        public void run() {
            for (BlockState state : blocks) {
                if (state.getType() == Material.OBSIDIAN) {
                    Block realBlock = state.getBlock();
                    realBlock.setType(Material.CRYING_OBSIDIAN);
                    realBlock.getWorld().playSound(realBlock.getLocation(), Sound.BLOCK_GLASS_BREAK, 1.0f, 0.5f);
                    realBlock.getWorld().spawnParticle(Particle.LAVA, realBlock.getLocation(), 30);
                    break;
                }
            }
        }
    }.runTaskLater(this, 1L);
}
```



Совет: Если код подчеркнут красным, нажмите на него и нажмите Ctrl + . , чтобы VS Code автоматически добавил нужный импорт

